

Clean a messy dataset in Pandas

handling missing values, filtering rows, creating new columns

```
In [555... import pandas as pd
```

```
In [556... df = pd.read_csv(r"data.csv")  
df
```

Out[556...

	Duration	Date	Pulse	Maxpulse	Calories
0	60	'2020/12/01'	110	130	409.1
1	60	'2020/12/02'	117	145	479.0
2	60	'2020/12/03'	103	135	340.0
3	45	'2020/12/04'	109	175	282.4
4	45	'2020/12/05'	117	148	406.0
5	60	'2020/12/06'	102	127	300.0
6	60	'2020/12/07'	110	136	374.0
7	450	'2020/12/08'	104	134	253.3
8	30	'2020/12/09'	109	133	195.1
9	60	'2020/12/10'	98	124	269.0
10	60	'2020/12/11'	103	147	329.3
11	60	'2020/12/12'	100	120	250.7
12	60	'2020/12/12'	100	120	250.7
13	60	'2020/12/13'	106	128	345.3
14	60	'2020/12/14'	104	132	379.3
15	60	'2020/12/15'	98	123	275.0
16	60	'2020/12/16'	98	120	215.2
17	60	'2020/12/17'	100	120	300.0
18	45	'2020/12/18'	90	112	NaN
19	60	'2020/12/19'	103	123	323.0
20	45	'2020/12/20'	97	125	243.0
21	60	'2020/12/21'	108	131	364.2
22	45	NaN	100	119	282.0
23	60	'2020/12/23'	130	101	300.0
24	45	'2020/12/24'	105	132	246.0
25	60	'2020/12/25'	102	126	334.5
26	60	20201226	100	120	250.0
27	60	'2020/12/27'	92	118	241.0
28	60	'2020/12/28'	103	132	NaN
29	60	'2020/12/29'	100	132	280.0

	Duration	Date	Pulse	Maxpulse	Calories
30	60	'2020/12/30'	102	129	380.3
31	60	'2020/12/31'	92	115	243.0

In [557... `df.shape`

Out[557... (32, 5)

In [558... `df.columns`

Out[558... Index(['Duration', 'Date', 'Pulse', 'Maxpulse', 'Calories'], dtype='str')

1) Checking Duplicate rows

In [559... `df[df.duplicated(keep=False)]` *# show all duplicate rows*

Out[559...

	Duration	Date	Pulse	Maxpulse	Calories
11	60	'2020/12/12'	100	120	250.7
12	60	'2020/12/12'	100	120	250.7

In [560... *# Drop duplicated rows*
`df = df.drop_duplicates()`
`df`

Out[560...

	Duration	Date	Pulse	Maxpulse	Calories
0	60	'2020/12/01'	110	130	409.1
1	60	'2020/12/02'	117	145	479.0
2	60	'2020/12/03'	103	135	340.0
3	45	'2020/12/04'	109	175	282.4
4	45	'2020/12/05'	117	148	406.0
5	60	'2020/12/06'	102	127	300.0
6	60	'2020/12/07'	110	136	374.0
7	450	'2020/12/08'	104	134	253.3
8	30	'2020/12/09'	109	133	195.1
9	60	'2020/12/10'	98	124	269.0
10	60	'2020/12/11'	103	147	329.3
11	60	'2020/12/12'	100	120	250.7
13	60	'2020/12/13'	106	128	345.3
14	60	'2020/12/14'	104	132	379.3
15	60	'2020/12/15'	98	123	275.0
16	60	'2020/12/16'	98	120	215.2
17	60	'2020/12/17'	100	120	300.0
18	45	'2020/12/18'	90	112	NaN
19	60	'2020/12/19'	103	123	323.0
20	45	'2020/12/20'	97	125	243.0
21	60	'2020/12/21'	108	131	364.2
22	45	NaN	100	119	282.0
23	60	'2020/12/23'	130	101	300.0
24	45	'2020/12/24'	105	132	246.0
25	60	'2020/12/25'	102	126	334.5
26	60	20201226	100	120	250.0
27	60	'2020/12/27'	92	118	241.0
28	60	'2020/12/28'	103	132	NaN
29	60	'2020/12/29'	100	132	280.0
30	60	'2020/12/30'	102	129	380.3

	Duration	Date	Pulse	Maxpulse	Calories
31	60	'2020/12/31'	92	115	243.0

In [561...

df['Date']

Out[561...

```

0    '2020/12/01'
1    '2020/12/02'
2    '2020/12/03'
3    '2020/12/04'
4    '2020/12/05'
5    '2020/12/06'
6    '2020/12/07'
7    '2020/12/08'
8    '2020/12/09'
9    '2020/12/10'
10   '2020/12/11'
11   '2020/12/12'
13   '2020/12/13'
14   '2020/12/14'
15   '2020/12/15'
16   '2020/12/16'
17   '2020/12/17'
18   '2020/12/18'
19   '2020/12/19'
20   '2020/12/20'
21   '2020/12/21'
22      NaN
23   '2020/12/23'
24   '2020/12/24'
25   '2020/12/25'
26      20201226
27   '2020/12/27'
28   '2020/12/28'
29   '2020/12/29'
30   '2020/12/30'
31   '2020/12/31'

```

Name: Date, dtype: str

2) Removing (')

at starting & at end if there

In [562...

```

df["Date"] = df["Date"].str.strip("")
df["Date"]

```

```
Out[562... 0    2020/12/01
           1    2020/12/02
           2    2020/12/03
           3    2020/12/04
           4    2020/12/05
           5    2020/12/06
           6    2020/12/07
           7    2020/12/08
           8    2020/12/09
           9    2020/12/10
          10    2020/12/11
          11    2020/12/12
          12    2020/12/13
          13    2020/12/14
          14    2020/12/15
          15    2020/12/16
          16    2020/12/17
          17    2020/12/18
          18    2020/12/19
          19    2020/12/20
          20    2020/12/21
          21    2020/12/21
          22         NaN
          23    2020/12/23
          24    2020/12/24
          25    2020/12/25
          26    20201226
          27    2020/12/27
          28    2020/12/28
          29    2020/12/29
          30    2020/12/30
          31    2020/12/31
          Name: Date, dtype: str
```

```
In [563... df
```

Out[563...

	Duration	Date	Pulse	Maxpulse	Calories
0	60	2020/12/01	110	130	409.1
1	60	2020/12/02	117	145	479.0
2	60	2020/12/03	103	135	340.0
3	45	2020/12/04	109	175	282.4
4	45	2020/12/05	117	148	406.0
5	60	2020/12/06	102	127	300.0
6	60	2020/12/07	110	136	374.0
7	450	2020/12/08	104	134	253.3
8	30	2020/12/09	109	133	195.1
9	60	2020/12/10	98	124	269.0
10	60	2020/12/11	103	147	329.3
11	60	2020/12/12	100	120	250.7
13	60	2020/12/13	106	128	345.3
14	60	2020/12/14	104	132	379.3
15	60	2020/12/15	98	123	275.0
16	60	2020/12/16	98	120	215.2
17	60	2020/12/17	100	120	300.0
18	45	2020/12/18	90	112	NaN
19	60	2020/12/19	103	123	323.0
20	45	2020/12/20	97	125	243.0
21	60	2020/12/21	108	131	364.2
22	45	NaN	100	119	282.0
23	60	2020/12/23	130	101	300.0
24	45	2020/12/24	105	132	246.0
25	60	2020/12/25	102	126	334.5
26	60	20201226	100	120	250.0
27	60	2020/12/27	92	118	241.0
28	60	2020/12/28	103	132	NaN
29	60	2020/12/29	100	132	280.0
30	60	2020/12/30	102	129	380.3

	Duration	Date	Pulse	Maxpulse	Calories
31	60	2020/12/31	92	115	243.0

3) Checking total null in all column

In [564... `df.isnull().sum()`

Out[564...
 Duration 0
 Date 1
 Pulse 0
 Maxpulse 0
 Calories 2
 dtype: int64

3) Replacing empty cell first with NaN.

Then output all null rows.

In [565... `import numpy as np`
`null_rows = df.replace(r'^\s*$', np.nan, regex=True)`
`null_rows = null_rows[null_rows.isnull().any(axis=1)]`
`null_rows`

Out[565...

	Duration	Date	Pulse	Maxpulse	Calories
18	45	2020/12/18	90	112	NaN
22	45	NaN	100	119	282.0
28	60	2020/12/28	103	132	NaN

4) Dropping NaN or Empty row

In [566... `import numpy as np`
`# Replace empty/whitespace-only strings with NaN, then drop all null rows`
`df = df.replace(r'^\s*$', np.nan, regex=True).dropna()`
`df`

Out[566...

	Duration	Date	Pulse	Maxpulse	Calories
0	60	2020/12/01	110	130	409.1
1	60	2020/12/02	117	145	479.0
2	60	2020/12/03	103	135	340.0
3	45	2020/12/04	109	175	282.4
4	45	2020/12/05	117	148	406.0
5	60	2020/12/06	102	127	300.0
6	60	2020/12/07	110	136	374.0
7	450	2020/12/08	104	134	253.3
8	30	2020/12/09	109	133	195.1
9	60	2020/12/10	98	124	269.0
10	60	2020/12/11	103	147	329.3
11	60	2020/12/12	100	120	250.7
13	60	2020/12/13	106	128	345.3
14	60	2020/12/14	104	132	379.3
15	60	2020/12/15	98	123	275.0
16	60	2020/12/16	98	120	215.2
17	60	2020/12/17	100	120	300.0
19	60	2020/12/19	103	123	323.0
20	45	2020/12/20	97	125	243.0
21	60	2020/12/21	108	131	364.2
23	60	2020/12/23	130	101	300.0
24	45	2020/12/24	105	132	246.0
25	60	2020/12/25	102	126	334.5
26	60	20201226	100	120	250.0
27	60	2020/12/27	92	118	241.0
29	60	2020/12/29	100	132	280.0
30	60	2020/12/30	102	129	380.3
31	60	2020/12/31	92	115	243.0

5) Resetting Index

```
In [567... df.reset_index(inplace= True)
df.head()
```

```
Out[567...
   index  Duration    Date  Pulse  Maxpulse  Calories
0      0        60 2020/12/01   110       130     409.1
1      1        60 2020/12/02   117       145     479.0
2      2        60 2020/12/03   103       135     340.0
3      3        45 2020/12/04   109       175     282.4
4      4        45 2020/12/05   117       148     406.0
```

```
In [568... df.columns
```

```
Out[568... Index(['index', 'Duration', 'Date', 'Pulse', 'Maxpulse', 'Calories'], dtype='str')
```

6) Keeping important columns

```
In [569... df2 = df[['Duration', 'Date', 'Pulse', 'Maxpulse', 'Calories']]
df2.head()
```

```
Out[569...
   Duration    Date  Pulse  Maxpulse  Calories
0        60 2020/12/01   110       130     409.1
1        60 2020/12/02   117       145     479.0
2        60 2020/12/03   103       135     340.0
3        45 2020/12/04   109       175     282.4
4        45 2020/12/05   117       148     406.0
```

7) Using a regex pattern

to find any 8-digit date string (like 20201201), format it with slashes (2020/12/01), and leave all other values completely untouched.

```
In [570... df2['Date'] = df2['Date'].replace(r'^(\d{4})(\d{2})(\d{2})$', r'\1/\2/\3', regex=True)
df2
```

Out[570...

	Duration	Date	Pulse	Maxpulse	Calories
0	60	2020/12/01	110	130	409.1
1	60	2020/12/02	117	145	479.0
2	60	2020/12/03	103	135	340.0
3	45	2020/12/04	109	175	282.4
4	45	2020/12/05	117	148	406.0
5	60	2020/12/06	102	127	300.0
6	60	2020/12/07	110	136	374.0
7	450	2020/12/08	104	134	253.3
8	30	2020/12/09	109	133	195.1
9	60	2020/12/10	98	124	269.0
10	60	2020/12/11	103	147	329.3
11	60	2020/12/12	100	120	250.7
12	60	2020/12/13	106	128	345.3
13	60	2020/12/14	104	132	379.3
14	60	2020/12/15	98	123	275.0
15	60	2020/12/16	98	120	215.2
16	60	2020/12/17	100	120	300.0
17	60	2020/12/19	103	123	323.0
18	45	2020/12/20	97	125	243.0
19	60	2020/12/21	108	131	364.2
20	60	2020/12/23	130	101	300.0
21	45	2020/12/24	105	132	246.0
22	60	2020/12/25	102	126	334.5
23	60	2020/12/26	100	120	250.0
24	60	2020/12/27	92	118	241.0
25	60	2020/12/29	100	132	280.0
26	60	2020/12/30	102	129	380.3
27	60	2020/12/31	92	115	243.0

MISTYPING

Removing extra 0 in 450.

In [571...

```
df2['Duration'] = df2['Duration'].astype(str).str.replace(r'(\d{2})\d+', r'\1', reg
df2

# pattern r'(\d{2})\d+' targets numbers with 3 or more digits and keep only first t
# Earlier at row 7 (Duration column) had 450 invalid value.
```

Out[571...

	Duration	Date	Pulse	Maxpulse	Calories
0	60	2020/12/01	110	130	409.1
1	60	2020/12/02	117	145	479.0
2	60	2020/12/03	103	135	340.0
3	45	2020/12/04	109	175	282.4
4	45	2020/12/05	117	148	406.0
5	60	2020/12/06	102	127	300.0
6	60	2020/12/07	110	136	374.0
7	45	2020/12/08	104	134	253.3
8	30	2020/12/09	109	133	195.1
9	60	2020/12/10	98	124	269.0
10	60	2020/12/11	103	147	329.3
11	60	2020/12/12	100	120	250.7
12	60	2020/12/13	106	128	345.3
13	60	2020/12/14	104	132	379.3
14	60	2020/12/15	98	123	275.0
15	60	2020/12/16	98	120	215.2
16	60	2020/12/17	100	120	300.0
17	60	2020/12/19	103	123	323.0
18	45	2020/12/20	97	125	243.0
19	60	2020/12/21	108	131	364.2
20	60	2020/12/23	130	101	300.0
21	45	2020/12/24	105	132	246.0
22	60	2020/12/25	102	126	334.5
23	60	2020/12/26	100	120	250.0
24	60	2020/12/27	92	118	241.0
25	60	2020/12/29	100	132	280.0
26	60	2020/12/30	102	129	380.3
27	60	2020/12/31	92	115	243.0

8) Saving Cleaned data to "Cleaned-data.xlsx" excel with sheet name - "Cleaned Data"

```
In [572... import os

excel_filename = "Cleaned-Data.xlsx"

if os.path.exists(excel_filename):
    with pd.ExcelWriter(excel_filename, engine='openpyxl', mode='a', if_sheet_exists='replace'):
        df2.to_excel(writer, index=False, sheet_name="Cleaned Data")
else:
    df2.to_excel(excel_filename, index=False, sheet_name="Cleaned Data")

print(f"Data successfully saved to {excel_filename}!")
```

Data successfully saved to Cleaned-Data.xlsx!

```
In [573... df2.to_csv('Cleaned Data.csv', index=False)
```

9) Checking dataTypes

```
In [574... df2.dtypes
```

```
Out[574... Duration      str
Date          str
Pulse         int64
Maxpulse      int64
Calories      float64
dtype: object
```

9) Grouping

Checking to see what's inside the column, since int64 means Pandas sees them as numbers:

```
In [575... print(df2['Duration'].unique())
print(df2['Duration'].value_counts())
```

```
<ArrowStringArray>
['60', '45', '30']
Length: 3, dtype: str
Duration
60    22
45     5
30     1
Name: count, dtype: int64
```

```
In [576... # Seeing rows whose values are 60
df2['Duration'] = df2['Duration'].astype(str).str.strip()
df2[df2['Duration'] == '60']
```

Out[576...

	Duration	Date	Pulse	Maxpulse	Calories
0	60	2020/12/01	110	130	409.1
1	60	2020/12/02	117	145	479.0
2	60	2020/12/03	103	135	340.0
5	60	2020/12/06	102	127	300.0
6	60	2020/12/07	110	136	374.0
9	60	2020/12/10	98	124	269.0
10	60	2020/12/11	103	147	329.3
11	60	2020/12/12	100	120	250.7
12	60	2020/12/13	106	128	345.3
13	60	2020/12/14	104	132	379.3
14	60	2020/12/15	98	123	275.0
15	60	2020/12/16	98	120	215.2
16	60	2020/12/17	100	120	300.0
17	60	2020/12/19	103	123	323.0
19	60	2020/12/21	108	131	364.2
20	60	2020/12/23	130	101	300.0
22	60	2020/12/25	102	126	334.5
23	60	2020/12/26	100	120	250.0
24	60	2020/12/27	92	118	241.0
25	60	2020/12/29	100	132	280.0
26	60	2020/12/30	102	129	380.3
27	60	2020/12/31	92	115	243.0

In [577...

```
# Count how many rows are in each duration group
df2.groupby('Duration').size()
```

Out[577...

```
Duration
30      1
45      5
60     22
dtype: int64
```

10) Just seeing before grouping

In [578...

```
for duration_val, group in df2.groupby('Duration'):
    print(f"--- Duration: {duration_val} ---")
```

```
display(group)
```

--- Duration: 30 ---

	Duration	Date	Pulse	Maxpulse	Calories
8	30	2020/12/09	109	133	195.1

--- Duration: 45 ---

	Duration	Date	Pulse	Maxpulse	Calories
3	45	2020/12/04	109	175	282.4
4	45	2020/12/05	117	148	406.0
7	45	2020/12/08	104	134	253.3
18	45	2020/12/20	97	125	243.0
21	45	2020/12/24	105	132	246.0

--- Duration: 60 ---

	Duration	Date	Pulse	Maxpulse	Calories
0	60	2020/12/01	110	130	409.1
1	60	2020/12/02	117	145	479.0
2	60	2020/12/03	103	135	340.0
5	60	2020/12/06	102	127	300.0
6	60	2020/12/07	110	136	374.0
9	60	2020/12/10	98	124	269.0
10	60	2020/12/11	103	147	329.3
11	60	2020/12/12	100	120	250.7
12	60	2020/12/13	106	128	345.3
13	60	2020/12/14	104	132	379.3
14	60	2020/12/15	98	123	275.0
15	60	2020/12/16	98	120	215.2
16	60	2020/12/17	100	120	300.0
17	60	2020/12/19	103	123	323.0
19	60	2020/12/21	108	131	364.2
20	60	2020/12/23	130	101	300.0
22	60	2020/12/25	102	126	334.5
23	60	2020/12/26	100	120	250.0
24	60	2020/12/27	92	118	241.0
25	60	2020/12/29	100	132	280.0
26	60	2020/12/30	102	129	380.3
27	60	2020/12/31	92	115	243.0

In [579... `df2.columns`

Out[579... `Index(['Duration', 'Date', 'Pulse', 'Maxpulse', 'Calories'], dtype='str')`

In [580... `df2[cols] = df2[cols].round(2)`

In [581... `df2.head()`

Out[581...

	Duration	Date	Pulse	Maxpulse	Calories
0	60	2020/12/01	110	130	409.1
1	60	2020/12/02	117	145	479.0
2	60	2020/12/03	103	135	340.0
3	45	2020/12/04	109	175	282.4
4	45	2020/12/05	117	148	406.0

In [582...

```
df3 = df2.groupby('Duration', as_index=False).mean(numeric_only=True)
df3
```

Out[582...

	Duration	Pulse	Maxpulse	Calories
0	30	109.000000	133.000000	195.100000
1	45	106.400000	142.800000	286.140000
2	60	103.545455	126.454545	317.359091

11) Rounding Number to 2 digit

In [583...

```
df3[cols] = df3[cols].round(2)
```

In [584...

```
df3.columns
```

Out[584...

```
Index(['Duration', 'Pulse', 'Maxpulse', 'Calories'], dtype='str')
```

In [585...

```
df3 = df3.rename(columns={'Duration': 'Duration(Grouped By)'})
df3
```

Out[585...

	Duration(Grouped By)	Pulse	Maxpulse	Calories
0	30	109.00	133.00	195.10
1	45	106.40	142.80	286.14
2	60	103.55	126.45	317.36

12) Saving in excel in different sheet as output

In [586...

```
import os, openpyxl, pandas as pd
from openpyxl.styles import Font

f = "Cleaned-data.xlsx"
mode = 'a' if os.path.exists(f) else 'w'

with pd.ExcelWriter(f, engine='openpyxl', mode=mode, if_sheet_exists='replace') as
    df3.to_excel(w, index=False, sheet_name="Pivot Table", startrow=1)
print(f"Data successfully saved to {excel_filename}!")
```

Data successfully saved to Cleaned-Data.xlsx!

13) Saving to CSV file

```
In [587... df3.to_csv('Pivot Table.csv', index=False)
```

```
In [ ]:
```